

ALGOVERSE

A Story-Driven RPG for Learning Data Structures and Algorithms

Project Report

Submitted By:

Name: Satyansh

Registration Number: _____

Roll Number: _____

Programme:

B.Tech Computer Science and Engineering

University:

Lovely Professional University

Academic Session: 2024–2028

DECLARATION

I, Satyansh, hereby declare that the project report entitled “Algoverse — A Story-Driven RPG for Learning Data Structures and Algorithms”, submitted in partial fulfilment of the requirements for the degree of B.Tech in Computer Science and Engineering at Lovely Professional University, is an original record of work carried out by me under the guidance of my project supervisor.

I further declare that the work presented in this report, including the design, implementation, and documentation of the Algoverse project, has not been submitted, in part or in full, to any other university or institution for the award of any other degree or diploma. Where information and material from other sources have been used, they have been duly acknowledged.

Place: _____

Date: _____

Signature: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor and the faculty of the Department of Computer Science and Engineering, Lovely Professional University, for their guidance, feedback, and support throughout the development of this project.

I am also thankful to my peers for their suggestions during informal reviews of the gameplay and code-execution systems, and to my family for their continued encouragement during the course of this work.

Finally, I acknowledge the open-source communities behind React, Phaser, Vite, Node.js, and the Piston code-execution project, whose tools and documentation made the implementation of AlgoVerse possible.

ABSTRACT

Conventional approaches to teaching Data Structures and Algorithms (DSA) rely heavily on static textbooks, recorded lectures, and isolated coding-judge platforms where a learner reads a description of an algorithm and is then asked to solve an abstract problem on it. This sequence — learn the algorithm, then solve a question — separates conceptual understanding from implementation and offers little intuitive or exploratory context for why a technique works. Many learners can recite the steps of an algorithm without being able to recognise the underlying pattern when it appears in a new problem.

Algoverse is an RPG-based educational application that restructures this sequence into a single interactive loop: the player explores a game world, encounters an obstacle that embodies an algorithmic problem, works through a physical, world-based puzzle that visually mirrors the algorithm's logic, and only afterward is asked to formalise that understanding by implementing the algorithm in an integrated Code Trial. The Code Trial submission is executed against a set of visible and hidden automated test cases through a remote code-execution backend, and a successful submission unlocks a permanent entry for that algorithm inside an in-game reference book called the Ancient Codex. Player progress — including story milestones, completed trials, and unlocked Codex entries — is preserved between sessions through a versioned local save system.

The current implementation, built with React, the Phaser 2D game framework, and Vite on the frontend and a lightweight Node.js HTTP service on the backend, includes a playable Dungeon region that introduces the player to the game's mechanics through a Binary Search trial, and a first open region, the Plains of Origins, whose first quest teaches the Two Pointer technique through the story device of the Soul-Bind Pillars. Both trials pair a physical, narrative-driven puzzle with a Code Trial that supports submissions in C++, Java, Python, and JavaScript, evaluated through a remote sandboxed execution provider (Piston, with optional Judge0 support).

This report documents the motivation, requirement analysis, system architecture, and current implementation of Algoverse, drawing a clear distinction between features that are implemented and verifiable in the project repository, features that are partially implemented, and features that remain part of the project's future scope, such as the planned Sliding Window, Prefix Sum, Binary Search on Answer, and Kadane's Algorithm quests within the Plains of Origins. The report also discusses the engineering challenges encountered in combining a tile-

based game engine with a web-based coding environment, and evaluates the educational rationale, current limitations, and possible directions for extending the project.

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Background

Data Structures and Algorithms form the foundation of computer science education and are a central component of software engineering placement assessments. Despite their importance, DSA is often perceived by students as one of the more difficult and abstract subjects in the curriculum, largely because most learning material presents algorithms as sequences of steps to memorise rather than as ideas that can be experienced and reasoned about.

1.2 Motivation

The motivation behind Algoverse comes from the observation that many learners, including the author, find it easier to internalise a technique when it is tied to a concrete, visual, and interactive scenario rather than to a written explanation alone. Story-driven and puzzle-based learning is well established in domains such as language learning and mathematics education; Algoverse applies a similar approach to DSA by embedding algorithms into an explorable RPG world.

1.3 Problem Statement

Existing DSA learning resources — textbooks, video lectures, and coding-judge platforms — are effective at presenting formal definitions and at testing implementation ability, but they generally do not provide an intermediate, exploratory step in which the learner can observe or manipulate the mechanics of an algorithm before being asked to code it. As a result, the transition from “reading about an algorithm” to “implementing it correctly” can feel abrupt, particularly for beginners.

1.4 Proposed Solution

Algoverse proposes a learning loop — Explore → Encounter Problem → Understand Through Gameplay → Solve a Physical Puzzle → Implement in a Code Trial → Pass Test Cases → Unlock the Concept in the Ancient Codex — that inserts a visual and interactive understanding step between problem exposure and implementation. Each algorithm is represented as a region-specific quest with its own narrative framing, physical puzzle mechanic, and Code Trial.

1.5 Objectives

- To design and implement a 2D RPG game world in which algorithmic concepts are represented as explorable regions, quests, and puzzles.
- To build an integrated Code Trial system that allows players to submit algorithm implementations in multiple programming languages and receive automated test-case feedback.
- To connect physical, gameplay-based problem solving with formal algorithmic implementation for at least one complete quest, demonstrated through the Two Pointer technique.
- To maintain a persistent, versioned save system that records story progress, completed trials, and unlocked Ancient Codex entries.
- To document the current implementation honestly, distinguishing completed systems from planned expansions.

1.6 Scope of the Project

The current scope of Algoverse, as implemented in the reviewed repository, covers a single playable Dungeon region containing a Binary Search trial, and the first quest of the Plains of Origins region, which teaches the Two Pointer technique through the Soul-Bind Pillars storyline. The scope explicitly excludes, at this stage, the remaining planned Plains of Origins quests (Sliding Window, Prefix Sum, Binary Search on Answer, and Kadane's Algorithm), multiplayer features, cloud-based save synchronisation, and any authentication or user-account system, none of which are present in the current codebase.

2. EXISTING SYSTEM VS PROPOSED SYSTEM

Conventional DSA learning resources can be grouped into three broad categories, each with a different strength and a different gap that Algoverse attempts to address.

Textbooks and lecture notes are effective at presenting rigorous definitions, correctness proofs, and complexity analysis, but they are static: the learner cannot manipulate the data or observe the algorithm's behaviour interactively, and the notation can be difficult to connect to real code.

Video lectures add a demonstrative element and can visualise pointer movement or recursion trees, but they remain a passive medium — the learner watches an author execute the algorithm rather than performing the steps themselves, which limits active recall.

Online coding-judge platforms (such as LeetCode-style websites) are strong at practising implementation and at validating correctness through automated test cases, but they typically present a problem as an isolated statement with no narrative or exploratory context, and they assume the learner already understands the underlying technique before attempting the problem.

Algoverse does not claim that these existing approaches are ineffective; rather, it targets the specific gap between conceptual exposure and implementation by inserting an interactive, visual, narrative-driven step in between. The comparison below summarises the difference in approach rather than a judgement of quality.

Aspect	Conventional Resources	Algoverse
Presentation of concept	Text or spoken explanation	Narrative framing within a game world
Learner interaction with the algorithm	Largely passive (reading/watching)	Physical, world-based puzzle interaction
Implementation practice	Isolated coding problem	Code Trial tied directly to the preceding puzzle
Feedback	Pass/Fail on a judge	Visible and hidden automated test cases with contextual framing
Progress tracking	Problem list / streak counters	Quest progression and Ancient Codex unlock system
Context between problems	Minimal or none	Continuous story and world connecting each concept

3. REQUIREMENT ANALYSIS

3.1 Functional Requirements

- The system shall allow the player to move a character through a tile-based game world using keyboard input, with collision against wall and obstacle layers.
- The system shall present dialogue sequences with NPC characters (the Grandmaster) to introduce story context and quest objectives.
- The system shall present a physical, in-world puzzle representing an algorithmic technique (frequency pillars for Two Pointers; a search trial for Binary Search).
- The system shall provide an in-game Code Trial overlay containing a code editor, a language selector, starter code, and a Run/Submit action.
- The system shall transmit a player's submitted source code to a backend service, execute it against a defined set of visible and hidden test cases, and return pass/fail results.
- The system shall record completion of a Code Trial and correspondingly unlock the related entry in the Ancient Codex.
- The system shall persist game progression (story flags, trial completion, Codex unlocks, and player position) between sessions using a versioned save format, and shall support starting a New Journey or resuming via Continue.
- The system shall allow the player to review unlocked Ancient Codex entries at any time from the Main Menu or the in-game Pause Menu.

3.2 Non-Functional Requirements

- Usability: menus, dialogue, and the Code Trial overlay should be navigable using both keyboard and mouse, with clear visual feedback on interactive elements.
- Responsiveness: the Code Trial submission flow should provide a visible “running” state and return a result without requiring a page reload.
- Modularity: game scenes, challenge definitions, and backend execution logic are separated into independent modules (scenes, a challenge registry, and an execution service) to keep responsibilities isolated.

- **Maintainability:** challenge content (metadata, starter code, and test cases) is defined in data files (JSON/JS objects) rather than hard-coded inline, so that new quests can be added by extending the challenge registry.
- **Reliability:** the backend validates request payloads (challenge ID, language, source length) before dispatching code for execution, and handles compilation errors, runtime errors, and timeouts as distinct, user-visible states.
- **Performance:** code execution requests are bounded by explicit timeouts (a 12-second client-side abort on the Piston path) to avoid the interface hanging on an unresponsive execution provider.

3.3 Hardware Requirements (Development / Testing)

Component	Minimum Specification
Processor	Dual-core CPU, 2.0 GHz or higher
RAM	8 GB (16 GB recommended for smoother Vite/Phaser dev builds)
Storage	Approximately 500 MB free space for project files, dependencies, and assets
Display	1366 × 768 or higher resolution
Network	Internet connection (required for the remote code-execution provider)

3.4 Software Requirements

Component	Requirement
Operating System	Windows, macOS, or Linux
Runtime	Node.js (for both the Vite frontend dev server and the backend service)
Package Manager	npm
Browser	A modern browser with Canvas/WebGL support (for Phaser rendering)
Version Control	Git and a GitHub account
Code Editor	Any editor with JavaScript/JSX support (e.g. VS Code)

4. TECHNOLOGY STACK

The technology stack below reflects the dependencies actually declared in the project's package.json files and the imports used in the source code, rather than the aspirational stack described in the project's early design documentation (see Section 11.1 for a note on this discrepancy).

4.1 Frontend

Technology	Purpose	Why Used
React 19	Screen/UI composition (menus, overlays, HUD)	Component-based UI and state management for non-game screens
Phaser 4	2D game engine for the playable scenes	Handles tilemaps, sprites, arcade physics, and scene lifecycle
Vite	Frontend build tool and dev server	Fast development server with a built-in proxy used to forward /api requests to the backend
JavaScript (ES Modules) / JSX	Application and game logic	Native language of the React and Phaser ecosystem used in this project
HTML / CSS	Static markup and styling	Base structure and custom styling for menus, dialogue, and the Codex UI
ESLint	Static code linting	Enforces consistent code style during development

4.2 Backend

Technology	Purpose	Why Used
Node.js	Backend runtime	Runs the HTTP server that receives Code Trial submissions
Native “node:http” module	HTTP server implementation	The backend is implemented directly on Node's built-in HTTP server; no web framework such as Express is declared as a dependency in the repository
nodemon	Development convenience	Restarts the backend server automatically during development (declared as a devDependency at the repository root)

4.3 Game Development

Technology	Purpose	Why Used
Phaser Tilemap system	Loading and rendering tile-based maps	Renders the Dungeon and Plains of Origins environments from tilemap data
Tiled-style tilemaps / tileset images	Level and map authoring	Map and tileset assets are organised under assets/maps/dungeon and assets/maps/plains-of-origins
Custom pixel-art assets	Characters, environments, UI	Original sprites, backgrounds, and UI art stored under the assets/ directory

4.4 Code Execution

Code submitted through the Code Trial is never executed on the Algoverse backend host itself. The backend constructs a language-specific test harness around the player's submission and forwards it to a remote sandboxed execution provider, configured through environment variables:

- Piston (default) — a public, free code-execution API (the example configuration in `.env.example` points to <https://emkc.org/api/v2/piston>).
- Judge0 — an alternative provider supported by the same backend module, selectable via the `CODE_EXECUTION_PROVIDER` environment variable, for either a self-hosted instance or a RapidAPI-hosted endpoint.

This design was a deliberate security decision: the backend comment in `executionService.js` explicitly states that it “never runs untrusted code locally on host process or machine.”

4.5 Development Tooling

- Git and GitHub for version control and source hosting.
- ESLint for linting the frontend codebase.

No database, container platform (e.g. Docker), cloud deployment configuration, or authentication service is present in the repository at the time of this review; these are not claimed as part of the current stack.

5. SYSTEM ARCHITECTURE

Algoverse follows a client–server architecture in which nearly all game logic, story state, and rendering occur in the browser-based frontend, while only algorithm-submission execution is delegated to a backend service, which in turn delegates the actual code execution to a third-party sandboxed provider.

5.1 High-Level Flow

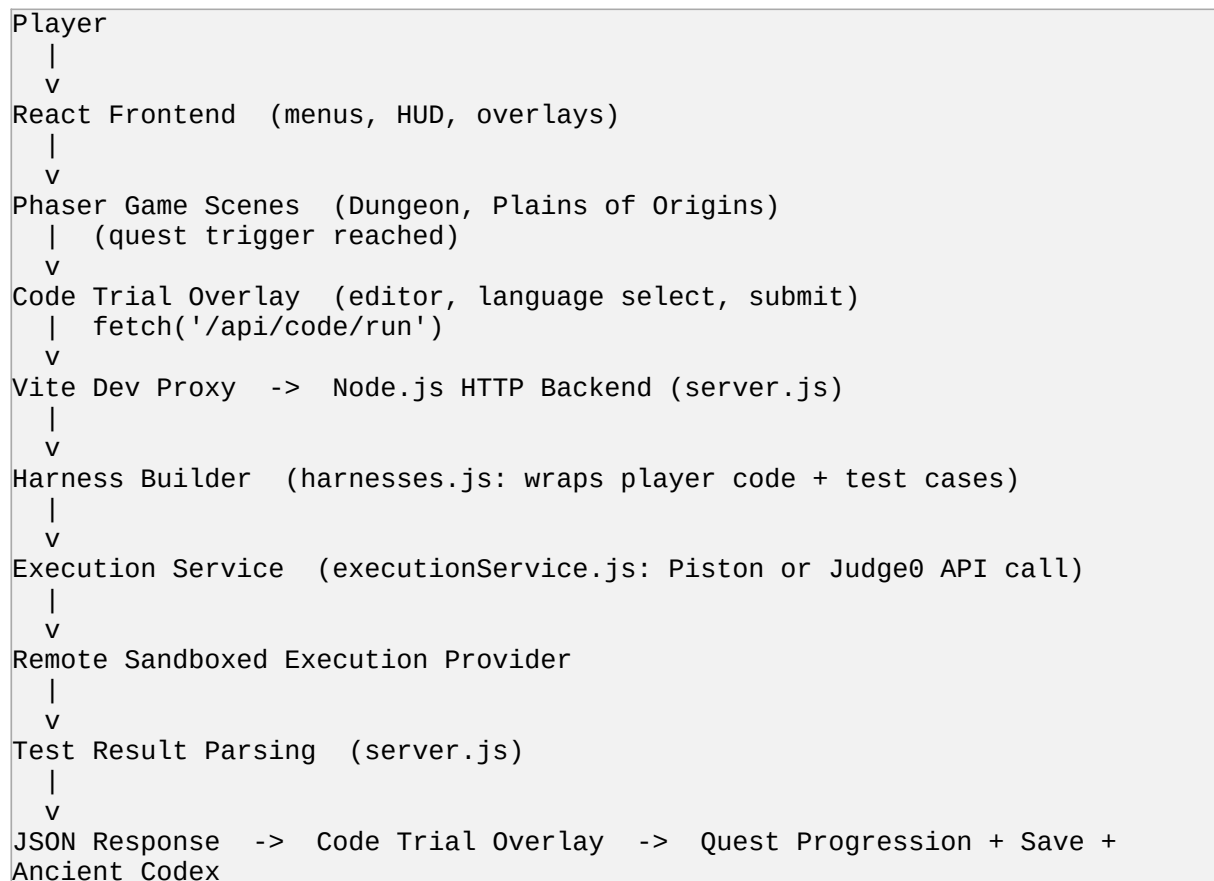


Figure A — Algoverse Request/Response Flow for a Code Trial Submission

5.2 Component Responsibilities

Frontend / Game Client

Built with React and Phaser. React manages the top-level screens (Splash, Main Menu, Settings, the Ancient Codex, the Pause Menu, and the Code Trial overlay), while Phaser Scene classes (`DungeonScene` and `PlainsOfOriginsScene`) manage the actual playable, tile-based game world, including player movement, collision, NPC interaction, and quest triggers.

Game / Quest Logic

Progression flags (for example `hasTriggeredSealedGate`, `binaryPhysicalTrialCompleted`, `gateUnlocked`, `plainsDiscovered`, `quest1Discovered`) are tracked as state within the React App component and passed down to the active scene, and are the basis for both save-game persistence and Ancient Codex unlock checks.

Code Trial Interface

The `CodeTrialOverlay` component provides a plain-text code editor (a styled `<textarea>`, not a full IDE component), a language selector (C++, Java, Python, JavaScript), and per-language starter code drafts. On submission, it calls the backend's `/api/code/run` endpoint.

Backend API

A single Node.js HTTP server (`backend/server.js`) exposes two endpoints: `GET /api/health` for a basic status check, and `POST /api/code/run`, which validates the request, loads the corresponding challenge's test suite from a JSON file, builds a language-specific harness around the submitted code, and requests execution.

Execution Service

`backend/executionService.js` abstracts over two possible remote providers (Piston and Judge0), translating a common `{language, sourceCode}` request into the provider-specific request format and normalising the response into a common shape (`SUCCESS`, `COMPILATION_ERROR`, `RUNTIME_ERROR`, or `TIME_LIMIT_EXCEEDED`).

Test Harness

`backend/harnesses.js` generates a complete, compilable/runnable program for each supported language by wrapping the player's function with a small runner that invokes it against every visible and hidden test case and prints a machine-parseable marker line (`[ALGOVERSE_TEST_RESULT:n]`) for each result.

Result Parsing and Quest Progression

The backend parses the marker lines out of the executed program's stdout, compares each result against the expected value, and returns a structured JSON response distinguishing visible results (shown in full to the player) from hidden results (pass/fail only, without revealing input or expected output). The frontend uses this response to update quest state, trigger the Ancient Codex unlock, and persist the change through the Save Manager.

5.3 Note on the System Architecture Diagram

The project's own design documentation (docs/game-design/GAME_ARCHITECTURE.md) describes a different high-level architecture — a React frontend communicating with a “C++ Backend” and a “DSA Algorithm Engine” through a REST API. This document appears to represent early or aspirational planning rather than the current implementation: the backend present in the repository is written in JavaScript on Node.js, and there is no C++ backend or standalone algorithm engine in the codebase. The architecture described in Sections 5.1 and 5.2 above reflects the code as it actually exists in the repository at the time of this review.

6. PROJECT DIRECTORY STRUCTURE

The directory layout below reflects the actual top-level structure of the Algoverse repository at the time of review.

```
Algoverse/
|-- assets/                               Shared game art and media
|   |-- background/                       Title screen and backdrop art
|   |-- characters/                       Character sprites (Hero, Grandmaster)
|   |-- cutscenes/                       Intro cutscene frames/media
|   |-- maps/
|       |-- dungeon/                     Dungeon tilesets and tilemaps
|       |-- plains-of-origins/           Plains tilesets, maps, quest-1 assets
|   |-- sprites/                         General sprite assets
|   |-- trials/                          Trial/puzzle-specific art
|   |-- ui/                              UI art (menus, panels, icons)
|-- backend/
|   |-- server.js                         Node.js HTTP server & API endpoints
|   |-- harnesses.js                     Per-language test harness builders
|   |-- executionService.js              Piston / Judge0 execution adapter
|-- docs/
|   |-- game-design/                     Design documentation (architecture, flow,
screens)
|-- frontend/
|   |-- index.html
|   |-- vite.config.js                   Vite config incl. /api proxy to backend
|   |-- eslint.config.js
|   |-- package.json
|   |-- src/
|       |-- App.jsx                     Top-level screen/state orchestration
|       |-- main.jsx                    React entry point
|       |-- challenges/                 Challenge registry, metadata, tests
|           |-- binary-search/
|           |-- two-pointers/
|           |-- index.js                 getChallengeById() loader
|       |-- game/
|           |-- config/                 Phaser game configuration
|           |-- events/                 Cross-component game event bus
|           |-- scenes/                 DungeonScene.js, PlainsOfOriginsScene.js
|       |-- pages/                      React screens (MainMenu, GameScreen,
CodeTrialOverlay, AncientCodex, etc.)
|       |-- services/
|           |-- saveManager.js           Versioned localStorage save/settings
|-- .env.example                         Code-execution provider configuration
|-- package.json                         Root scripts (backend, frontend)
|-- README.md
```

Figure B — Algoverse Repository Directory Structure

Two directories referenced in the project's design documentation — `scripts/` and `.github/` — are described as planned utility/workflow locations in

docs/game-design/GAME_ARCHITECTURE.md but were not present as populated directories in the reviewed repository snapshot.

7. GAME DESIGN

7.1 Game Concept

Algoverse frames each Data Structure or Algorithm as a location, obstacle, or artefact within a fantasy world named Algorithmia. Rather than presenting an algorithm as an abstract procedure, the game embeds it into a story problem the Hero must physically resolve before being asked to formalise the same logic as code.

7.2 Hero

The player controls a customisable Hero sprite that moves through each region using keyboard input, interacting with NPCs, rune systems, and quest triggers. The Hero has no combat statistics or inventory system implemented at this stage; interaction is limited to movement, dialogue triggers, and puzzle/quest mechanics.

7.3 Grandmaster

The Grandmaster is the Hero's guide and mentor character, introduced within the Dungeon. The Grandmaster delivers story and instructional dialogue before and after each trial, framing the algorithmic challenge in narrative terms and prompting the player to “encode” what they have just learned once the physical puzzle is solved.

7.4 World Structure

The implemented world currently consists of two regions: the Dungeon, which serves as an introductory area, and the Plains of Origins, the first major open region reached after the Dungeon. The Plains of Origins is designed to host multiple algorithm-based quest zones, of which one (Two Pointers / Soul-Bind Pillars) is implemented.

7.5 Exploration

Exploration is implemented through Phaser tilemap scenes with arcade-physics collision against wall layers, allowing the Hero to walk through defined paths, trigger zones (such as the sealed gate in the Dungeon), and transition between regions.

7.6 Dialogue System

Dialogue is presented through scripted sequences tied to specific triggers (for example, meeting the Grandmaster or approaching the Soul-Bind Pillars), advancing the story and providing instructional framing for the upcoming puzzle.

7.7 Quest Structure

Each implemented quest follows the same repeating structure: a story trigger introduces the obstacle, a physical puzzle lets the player interact with a visual representation of the algorithm, a Code Trial then asks the player to implement the same logic formally, and successful completion updates quest-progression flags, unlocks the associated Ancient Codex entry, and is persisted to the save file.

7.8 Algorithm Learning Loop

Discover	->	Understand	->	Interact	->	Solve	->	Code	->	Verify	->
Unlock	->	Progress									

This loop is the structural backbone of both implemented trials (Binary Search in the Dungeon and Two Pointers in the Plains of Origins) and is intended to be reused for each future Plains of Origins quest.

8. GAME FLOW

The flow below reflects the actual screen/state transitions implemented in the top-level App component, cross-checked against the design intent in docs/game-design/PLAYER_FLOW.md and SCREENS.md.

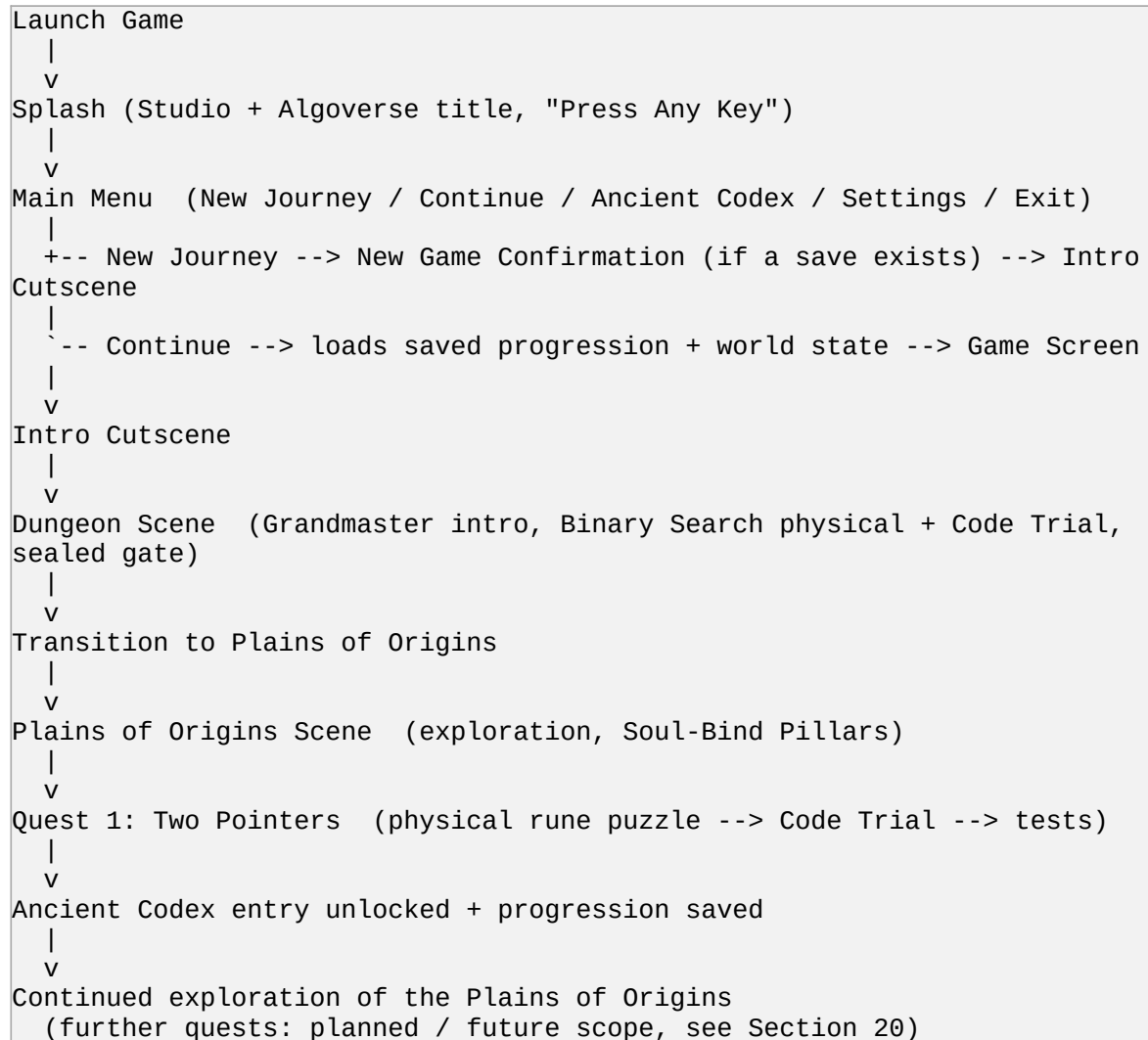


Figure C — Implemented and Near-Term Game Flow

The project's design documentation additionally describes further downstream screens — an Inventory screen, a “Save Shrine,” a Victory Screen, and a Game Over screen — as part of the intended full player flow. None of these four screens were found implemented as functioning React/Phaser components in the reviewed codebase; they are treated in this report as future scope (Section 20) rather than current functionality.

9. DUNGEON MODULE

9.1 Purpose

The Dungeon is the game's introductory region. It is implemented as a dedicated Phaser scene (DungeonScene.js) and is responsible for teaching the player the game's core interaction patterns — movement, dialogue, and the Code Trial flow — before the player is given access to the larger, more open Plains of Origins region.

9.2 Story Introduction

On entering the Dungeon, the player meets the Grandmaster, who introduces the setting and briefs the Hero on their objective. Progression flags such as `hasCompletedGrandmasterIntro` and `hasCompletedBriefing` gate subsequent story beats.

9.3 Environment

The Dungeon environment is built from a Phaser tilemap (loaded under the key "dungeon-01") combined with separate tileset images for the main environment and decorative layers, with a dedicated static physics layer used purely for arcade collision against walls.

9.4 Player Interaction and Algorithmic Trial

The Dungeon's algorithmic trial is Binary Search, tracked through the flags `binaryPhysicalTrialCompleted` and `binaryCodeTrialCompleted`. The player engages with a rune-based physical trial (referencing binary-rune art assets loaded by the scene) before being directed to the Code Trial to implement the search function formally.

9.5 Coding Terminal

Reaching the coding stage of the Dungeon trial opens the same `CodeTrialOverlay` component used throughout the game, pre-configured with the “binary-search” challenge ID, so the player implements a function named `search(arr, target)` that must return the index of a target value in a sorted array, or -1 if the value is absent.

9.6 Progression and the Sealed Gate

A “sealed gate” trigger zone in the Dungeon is only unlocked once the necessary progression flags (including `gateUnlocked`) are set, following completion of the Binary Search trial and a follow-up dialogue beat with the Grandmaster (`grandmasterPostTrialDone`).

9.7 Transition to the Plains of Origins

Once the gate is unlocked and the player reaches the Dungeon's exit trigger, the game transitions into the Plains of Origins scene, setting the `plainsDiscovered` flag and updating the persisted `world.scene` value in the save file to reflect the new active region.

10. PLAINS OF ORIGINS

10.1 Environment

The Plains of Origins is implemented as a second Phaser scene (`PlainsOfOriginsScene.js`) with its own tilemap and tileset assets, stored separately from the Dungeon under `assets/maps/plains-of-origins/`. It represents the first open-world-style region the player reaches after escaping the Dungeon.

10.2 Map Structure

The Plains map assets include a dedicated quest-1 subdirectory (`assets/maps/plains-of-origins/quest-1/`), indicating that quest-specific art and map data are organised separately from the general region tileset, consistent with a design intended to support multiple quest zones within the same region over time.

10.3 Progression and Barriers

The Plains of Origins introduces a narrative barrier — a magical resonance field maintained by the Soul-Bind Pillars — that blocks the Hero's further progress through the region until Quest 1 is solved. This barrier mechanic is the region's implementation of a “quest gate,” structurally similar to the sealed gate in the Dungeon.

10.4 Quest Regions

At the time of this review, one quest region is implemented within the Plains of Origins: the Soul-Bind Pillars (Two Pointers). The project's README and design documentation describe four additional planned quest regions within the same Plains of Origins map, discussed as future scope in Section 20.

10.5 Connection Between Algorithms and World Mechanics

The Plains of Origins reuses the same learning loop established in the Dungeon: exploration leads to an obstacle, the obstacle is expressed as a physical, rune-based or environmental puzzle, and the puzzle's solution must then be reproduced as working code in the Code Trial before the world state changes (the barrier falls, the region unlocks further, and the Ancient Codex is updated).

11. QUEST 1 — TWO POINTERS

11.1 Quest Introduction

Quest 1 of the Plains of Origins is titled, in the project's own documentation, “Disarming the Soul-Bind Pillars.” It is the first fully implemented quest of the open-world region and teaches the Two Pointer technique.

11.2 Resonance Barrier

The story frames the obstacle as a magical resonance field that prevents the Hero from progressing further into the Plains. The field is sustained by two ancient pillars, giving the player a concrete, spatial metaphor for the two “ends” that the Two Pointer technique manipulates.

11.3 Rune / Pillar System

The two pillars hold rune stones containing sorted harmonic frequency values. A target frequency is generated for the encounter, and the Hero must find two frequencies whose sum equals that target in order to destabilise the barrier.

11.4 Frequency Values

The frequency values used in the physical puzzle correspond directly to the sorted integer array used later in the Code Trial's test cases (`challenges/two-pointers/tests/tests.json`), so the same numeric scenario the player manipulates physically is the scenario their code must later solve correctly.

11.5 Left and Right Pointers

The player interacts with the rune system using two reference points: one beginning at the lowest frequency and one at the highest. The pointers move inward based on whether the current combined value is below, above, or equal to the target — a direct, physical enactment of the Two Pointer algorithm before any code is written.

11.6 Physical Puzzle

Because the sum of the two selected frequencies determines whether the near pointer or the far pointer should move, the puzzle mechanically reproduces the decision rule of the Two Pointer technique, allowing the player to discover the pattern experimentally rather than being told the rule directly.

11.7 Algorithmic Interpretation

The Two Pointer technique operates on a sorted array by maintaining two indices, one at each end, and narrowing the search space based on a comparison against a target value. For the problem of finding a pair with a given sum in a sorted array, the technique runs in linear time and constant extra space, in contrast to a brute-force pairwise comparison, which requires quadratic time.

```
left = 0
right = n - 1

while left < right:
    sum = arr[left] + arr[right]

    if sum == target:
        pair found

    else if sum < target:
        left = left + 1

    else:
        right = right - 1
```

Time Complexity: $O(n)$ — each iteration moves the left pointer forward or the right pointer backward, and the two pointers can move at most n times combined before they meet.

Space Complexity: $O(1)$ — the technique uses a constant amount of extra memory beyond the input array, regardless of its size.

11.8 Code Trial

After solving the physical puzzle, the Grandmaster asks the Hero to “encode” the technique. The Code Trial overlay opens with the “two-pointers” challenge loaded, and the player implements a function named `findResonancePair(frequencies, target)`, which must return an array containing the two matching values, or an empty array if no such pair exists.

Starter code is provided per language. The C++ starter, for example, is:

```
vector<int> findResonancePair(const vector<int>& frequencies, int target) {
    // Write your Two Pointer solution here
}
```

11.9 Test Case Evaluation

The challenge defines five visible test cases (shown to the player with input, expected output, and the actual received value) and six additional hidden test cases (evaluated as pass/fail only,

without revealing input or expected values to the player). The visible cases cover a mix of positive, negative, and mixed-sign frequency values as well as a case with no valid pair; the hidden cases add edge conditions such as adjacent-pair matches at either end of the array, sums below the minimum possible pair, sums above the maximum possible pair, and a target of zero.

11.10 Quest Completion

A submission is only considered a full success once every visible test and every hidden test passes (allPassed is computed as the logical AND of all results). On success, the backend response includes the message “ALL TESTS PASSED — The path has been forged.”

11.11 Ancient Codex Unlock

On a passing submission, the frontend updates progression state (quest1Data, and the derived twoPointersUnlocked / quest1Solved flags checked by the Ancient Codex component) so that a Two Pointers entry becomes visible the next time the player opens the Codex.

11.12 Educational Outcome

By requiring the player to physically enact the pointer-narrowing rule before writing any code, Quest 1 aims to make the eventual implementation feel like a formalisation of an already-understood idea rather than the introduction of an unfamiliar procedure.

12. CODE TRIAL SYSTEM

12.1 Editor

The Code Trial interface is implemented as a React overlay component (CodeTrialOverlay.jsx). The code editor itself is a styled, plain HTML `<textarea>` rather than a full syntax-highlighting IDE component; it includes basic editing convenience such as inserting four spaces on Tab key press instead of shifting focus.

12.2 Language Selection

A language selector allows the player to choose between C++, Java, Python, and JavaScript. C++ is the default selected language when the overlay opens.

12.3 Starter Code

Each supported language has its own starter code template defined per challenge in the challenge registry (frontend/src/challenges/index.js), and the player's in-progress code for each language is retained in local component state (drafts) so switching languages does not discard unsaved work within the same session.

12.4 Submission and API Communication

On submission, the frontend issues a POST request to the relative path `/api/code/run` with the challenge ID, selected language, and source code. During local development, Vite's dev-server proxy configuration forwards any request under `/api` to the backend running on `http://localhost:3001`.

12.5 Backend Processing

The backend validates the payload, loads the appropriate challenge's test suite (visible and hidden test cases) from its JSON definition file, and builds a complete runnable program by wrapping the player's submitted function inside a language-specific harness (see Section 13).

12.6 Execution and Result Mapping

The wrapped program is sent to the configured remote execution provider. The backend inspects the returned exit status, compiler output, and stdout to classify the outcome as `SUCCESS`, `COMPILATION_ERROR`, `RUNTIME_ERROR`, or `TIME_LIMIT_EXCEEDED`, and, for a successful run, parses the harness's marker output to determine which individual test cases passed.

12.7 Expected vs. Actual Output

For visible test cases, the response returned to the frontend includes the test's input, expected value, and the value actually received from the player's code, along with a boolean passed flag, allowing the Code Trial overlay to display a clear per-case comparison.

12.8 Success / Failure States

The overlay distinguishes between an unconfigured execution backend (`isConfigured: false`, shown when no `CODE_EXECUTION_URL` is set), a configured backend that could not be reached, and a configured backend that successfully executed the code but reported one or more failing tests.

12.9 Timeout Handling

Two independent timeout mechanisms are present: the frontend's request to the execution provider is capped by an `AbortController` with a 12-second limit on the Piston path (30 seconds on the Judge0 path), and the execution provider itself enforces compile and run timeouts (10 seconds compile / 5 seconds run for Piston; a 5-second CPU time limit for Judge0). A `SIGKILL/SIGTERM` signal or an aborted fetch is mapped to a `TIME_LIMIT_EXCEEDED` status shown to the player.

12.10 Separation of Player Code from Reference Solutions

The challenge registry deliberately separates player-facing data (starter templates and visible tests, returned by `getChallengeById()`) from the full test suite, including hidden tests, which is only read server-side from the challenge's `tests/tests.json` file. The two-pointers challenge module also contains a `solutions.js` file that is not part of the payload returned to the frontend, keeping a reference implementation out of the client bundle.

13. MULTI-LANGUAGE EXECUTION

The system currently supports exactly four languages, consistently enforced on both the frontend (starter templates and the language selector) and the backend (the `validLangs` allow-list in `server.js`): C++, Java, Python, and JavaScript.

Language	Piston Runtime	Judge0 Language ID	Harness Entry Point
C++	c++ 10.2.0	54 (GCC 9.2.0)	Free function inside a generated <code>main()</code>
Java	java 15.0.2	62 (OpenJDK 13.0.1)	Reflection-based lookup of a free method or a <code>Solution</code> class
Python	python 3.10.0	71 (Python 3.8.1)	Global function or <code>Solution</code> class, detected at runtime
JavaScript	javascript 18.15.0	63 (Node.js 12.14.0)	Global function or <code>Solution</code> class, detected via <code>typeof</code>

For every language, `harnesses.js` generates two distinct harness families — one for the Binary Search challenge (calling `search(arr, target)`) and one for the Two Pointers challenge (calling `findResonancePair(frequencies, target)`) — selected based on the `challengeId` passed to `buildHarness()`. Each harness serialises the applicable test cases (visible and hidden combined) directly into the generated program and prints a uniquely tagged result line per test case, which the backend later parses with a regular expression.

Both the Java and JavaScript/Python harnesses are written to accept either a free-standing function definition or a LeetCode-style `Solution` class containing the same method, so that a player's code is not rejected purely because of a stylistic choice about how the function is declared.

14. ANCIENT CODEX

14.1 Purpose

The Ancient Codex is Algoverse's in-game reference book: a persistent, growing record of the algorithms the player has mastered over the course of the game, implemented as a single, reusable React component (`AncientCodex.jsx`) rendered as a full-screen overlay.

14.2 Algorithm Knowledge Storage

The Codex component resolves which entries to display by merging two sources: the live `progressionState` passed in from an active game session, and the persisted progression object loaded directly from `SaveManager`, so the Codex shows the correct unlocked state whether it is opened mid-session or from the Main Menu with no active session running.

14.3 Unlock System

Two entries are checked explicitly in the component: a Binary Search entry, gated by `binaryCodeTrialCompleted`, and a Two Pointers entry, gated by a combination of `twoPointersUnlocked`, `quest1Solved`, and their saved-data equivalents.

14.4 Relation to Quest Completion

An entry only appears once its associated Code Trial has been passed in full — the physical puzzle alone does not unlock a Codex entry, which reinforces the project's stated principle that “only after successfully passing the coding trial is the algorithm considered mastered.”

14.5 Two Pointers Entry

The Two Pointers entry becomes available once Quest 1 in the Plains of Origins has been completed, consistent with the quest structure described in Sections 10 and 11.

14.6 Educational Usefulness

As a design pattern, the Codex functions as a lightweight, self-curated study reference: because each entry is only unlocked after both a conceptual (physical) and a formal (coding) demonstration of understanding, the growing Codex content doubles as a visible record of what the player can confidently claim to know.

15. SAVE AND PROGRESSION SYSTEM

15.1 Implementation

Persistence is implemented entirely client-side through a dedicated `SaveManager` module (`frontend/src/services/saveManager.js`) that wraps the browser's `localStorage` API. There is no server-side database or account system; save data is local to the browser profile in which the game is played.

15.2 What Is Persisted

The save payload is versioned (version: 1) and contains a timestamp (`savedAt`), a progression object, and a world object:

- Story/progression flags: `hasTriggeredSealedGate`, `hasCompletedGrandmasterIntro`, `hasCompletedBriefing`, `binaryPhysicalTrialCompleted`, `binaryCodeTrialCompleted`, `grandmasterPostTrialDone`, `gateUnlocked`, `plainsDiscovered`, `quest1Discovered`, and `quest1Data`.
- World state: the active scene key (defaulting to `"DungeonScene"`) and the player's last recorded x/y position within that scene.

Ancient Codex unlock state is not stored as a separate field; it is derived at read-time from the progression flags described above (see Section 14.2), so it is implicitly persisted through the same save object.

15.3 Defensive Loading

`SaveManager.loadGame()` validates the shape of any data read from `localStorage` — checking the version number and the presence of both the progression and world sub-objects — and returns null if the stored data is missing, corrupted, or from an incompatible schema version, rather than allowing malformed data to reach the rest of the application.

15.4 Save Slots and New Journey / Continue

The system supports a single save slot per browser. Starting a “New Journey” when a save already exists prompts a confirmation dialog before `SaveManager.deleteSave()` clears the existing data and progression state is reset to its initial values. “Continue” loads the existing save and restores both `progressionState` and the last active scene.

15.5 Settings Persistence

Audio settings (masterVolume, musicVolume, sfxVolume) are stored separately from game progression, under their own localStorage key, and are merged with sensible defaults on load so a missing or partial settings object does not break playback configuration.

16. COLLISION AND MAP SYSTEM

16.1 Tilemaps

Both playable scenes load a tilemap (registered in Phaser's tilemap cache under a scene-specific key, e.g. "dungeon-01") together with one or more tileset images — a main environment tileset and a separate decorative tileset in the Dungeon's case — which Phaser combines at render time into the visible map.

16.2 Layers

Each map separates visual layers (ground, decoration) from a dedicated collision layer. The Dungeon scene explicitly comments a “Walls static layer — used for arcade collision,” indicating that collidable geometry is authored as its own tilemap layer rather than being inferred from visual tiles.

16.3 Object Layers and Trigger Regions

Story and quest triggers — such as the Dungeon's sealed gate and the Plains of Origins' resonance barrier — are implemented as trigger zones tied to specific map regions or objects, checked against the player's position each frame (or on overlap) to fire the corresponding story/quest event exactly once (guarded by boolean flags such as `gateTriggerActive`).

16.4 Collision Handling

Player-versus-wall collision uses Phaser's Arcade Physics system, with the player's sprite/physics body checked against the static walls layer so the Hero cannot walk through solid geometry.

16.5 Player Spawn and Area Transitions

On loading a scene, the player's spawn position is either set to a default entry point or restored from the saved world.player coordinates if a save exists for that scene, and moving into a defined exit/transition trigger hands control over to the next scene (for example, from the Dungeon to the Plains of Origins).

17. IMPORTANT ALGORITHMS / LOGIC

17.1 Two Pointer Reference Logic (Quest 1)

The core algorithmic logic taught by Quest 1 is the linear-time, sorted-array Two Pointer pattern described in Section 11.7; the player's own implementation of this logic inside `findResonancePair()` is what the backend harness and hidden test suite validate.

17.2 Result-Marker Parsing

A key piece of backend logic is the shared convention used across every language harness: each test case prints a line of the exact form `[ALGOVERSE_TEST_RESULT:<index>] <value>`, which the backend locates using a per-index regular expression and parses as JSON (falling back to a raw string/number) before comparing it against the expected value with a `JSON.stringify` equality check.

```
const regex = new RegExp(`\\[ALGOVERSE_TEST_RESULT:${i}\\]\\s*(.+)`);
const match = stdout.match(regex);
// ... JSON.parse(match[1]) with a safe fallback, then
passed = JSON.stringify(received) === JSON.stringify(t.expected);
```

17.3 Quest State Management

Progression is modelled as a flat set of boolean/plain-value flags held in React state at the App level and passed down into the active Phaser scene, updated through a single `handleProgressionUpdate` callback that merges partial updates into the existing state and immediately refreshes the cached `hasSave` flag.

17.4 Code Execution Pipeline

The pipeline — request validation, challenge/test lookup, harness construction, remote execution, and result parsing — is implemented as a linear sequence of function calls inside the `POST /api/code/run` handler in `server.js`, described fully in Section 12.

17.5 Save-State Handling

Save/load logic centres on the version-checked read/write pair in `SaveManager` (Section 15.3), which is the single place in the codebase responsible for guaranteeing that malformed or outdated save data cannot silently corrupt the running game's state.

18. TESTING

No automated test suite (unit tests, integration tests, or end-to-end tests) targeting the frontend or backend code itself was found in the repository. The table below reflects manual verification performed against the running application and the source code, not results from an automated test runner; entries are marked accordingly.

Test ID	Module	Test Scenario	Expected Result	Status
T-01	Player Movement	Move Hero using keyboard input in Dungeon/Plains	Hero sprite moves and animates in the pressed direction	Manual — Pass
T-02	Collision	Walk Hero into a walls-layer tile	Hero is blocked and cannot pass through	Manual — Pass
T-03	Dialogue Sequencing	Trigger Grandmaster intro dialogue	Dialogue sequence plays in defined order	Manual — Pass
T-04	Code Execution (valid)	Submit a correct findResonancePair() in C++	All visible and hidden tests pass; success message shown	Manual — Pass (with configured execution provider)
T-05	Code Execution (invalid)	Submit an incorrect solution	Failing visible tests are shown with expected vs. received values	Manual — Pass
T-06	Compilation Error	Submit C++ code with a syntax error	COMPILATION_ERROR status returned with compiler output	Manual — Pass
T-07	Timeout / TLE	Submit code with an infinite loop	TIME_LIMIT_EXCEEDED status returned within the configured limit	Manual — Pass
T-08	Unconfigured Backend	Submit code with no CODE_EXECUTION_URL set	isConfigured: false returned with an explanatory message	Manual — Pass
T-09	Save / Load	Complete Quest 1, reload the browser, click Continue	Progression, scene, and player position are restored	Manual — Pass
T-10	Corrupted Save Handling	Manually corrupt the localStorage save entry	loadGame() returns null instead of throwing	Manual — Pass
T-11	Ancient Codex Unlock	Complete the Two Pointers Code Trial	Two Pointers entry becomes visible in the Codex	Manual — Pass
T-12	Quest Progression Gate	Attempt to bypass the resonance barrier before solving Quest 1	Barrier remains active until the quest is completed	Manual — Pass

Because these results are based on manual, single-developer verification rather than an automated CI test suite, they should be read as a proposed verification record rather than a formally audited QA report.

19. CHALLENGES FACED

- Integrating Phaser's imperative, frame-based scene model with React's declarative component model required careful management of when a scene is mounted, torn down, and re-synchronised with progression state owned by React (e.g. passing `progressionState` into a scene and reading updates back out through a shared event bus).
- Mapping tileset and layer data correctly between the Dungeon and Plains of Origins scenes, given that each region uses its own tilemap key, tileset images, and collision layer, required consistent, scene-specific asset loading logic to avoid cross-region asset leakage.
- Defining reliable collision boundaries on a static walls layer that must line up precisely with the visual tile art, so that the Hero neither clips through walls nor is blocked by tiles that are only decorative.
- Coordinating scene transitions (Dungeon → Plains of Origins) so that trigger-based events fire exactly once, using explicit guard flags (e.g. `gateTriggerActive`, `isTransitioningToPlains`) rather than relying purely on physics-overlap events, which can otherwise fire multiple times per frame.
- Maintaining a single, consistent source of truth for quest state across three different consumers — the active Phaser scene, the React App component, and the persisted save file — without those three copies drifting out of sync.
- Designing the Two Pointers physical puzzle so that its interaction model (choosing a value from the low end or the high end of a sorted list) maps unambiguously onto the underlying algorithm, rather than reading as an arbitrary game mechanic loosely themed around the concept.
- Building a code-execution pipeline that supports four languages with two different structural conventions (free function vs. Solution class) using a single, shared JSON test-case format per challenge, without needing per-language test data.
- Handling the many distinct failure modes of remote code execution — compilation errors, runtime exceptions, timeouts, and an unreachable or unconfigured execution provider — as clearly separated states so the player receives an accurate, specific explanation rather than a generic failure message.

- Keeping the reference solution and hidden test cases out of the payload sent to the browser, since the Code Trial's test-case data and any exposed reference implementation live in the same directory tree as the player-facing metadata and starter code.
- Keeping game UI (dialogue boxes, HUD elements, overlays) legible when rendered on top of a busy, pixel-art tilemap background, which required a distinct, parchment/wood-themed UI style rather than a generic overlay style.

20. SCREENSHOTS

The figures below are placeholders for screenshots to be captured from a running instance of the application. No screenshot image files were found within the reviewed repository, so placeholders are used rather than fabricated images.

Figure 1 — Main Menu

[INSERT SCREENSHOT HERE]

Figure 2 — Dungeon

[INSERT SCREENSHOT HERE]

Figure 3 — Dungeon Code Trial

[INSERT SCREENSHOT HERE]

Figure 4 — Plains of Origins

[INSERT SCREENSHOT HERE]

Figure 5 — Resonance Barrier

[INSERT SCREENSHOT HERE]

Figure 6 — Soul-Bind Pillars

[INSERT SCREENSHOT HERE]

Figure 7 — Two Pointer Frequency Puzzle

[INSERT SCREENSHOT HERE]

Figure 8 — Two Pointer Code Trial

[INSERT SCREENSHOT HERE]

Figure 9 — Ancient Codex

[INSERT SCREENSHOT HERE]

Figure 10 — Quest Progression / Save System

[INSERT SCREENSHOT HERE]

21. CURRENT DEVELOPMENT STATUS

21.1 Completed

- Core RPG game foundation (Phaser scenes, tilemap rendering, arcade-physics collision)
- Main Menu, Splash flow, Settings panel, Exit and New Game confirmation dialogs
- Dungeon environment, Grandmaster introduction dialogue, and sealed-gate progression trigger
- Binary Search physical trial and Code Trial in the Dungeon
- Transition from the Dungeon to the Plains of Origins
- Plains of Origins environment and the Soul-Bind Pillars barrier mechanic
- Quest 1: Two Pointers — physical rune puzzle and Code Trial, including hidden test cases
- Code Trial overlay with a four-language editor, per-language starter code, and submission flow
- Backend HTTP server with a Piston/Judge0-backed execution service and per-language harness generation
- Ancient Codex overlay with unlock logic for the Binary Search and Two Pointers entries
- Versioned local save/continue system and separate audio settings persistence

21.2 Partially Implemented

- The Plains of Origins region is playable and contains one complete quest, but is designed to host four further quest zones that are not yet built.
- Code execution depends on an external, developer-configured provider (Piston or Judge0); the feature works end-to-end when configured, but is not bundled with a self-contained execution backend.

21.3 Planned (Not Yet Implemented)

- The remaining Plains of Origins quests: Sliding Window, Prefix Sum, Binary Search on Answer, and Kadane's Algorithm.
- Screens referenced in the project's design documentation but not found implemented: an Inventory screen, a Save Shrine, a Victory Screen, and a Game Over screen.

- Automated test coverage (unit/integration/end-to-end) for the frontend and backend codebases.

22. FUTURE SCOPE

The project's own documentation lays out a clear roadmap for expanding the Plains of Origins with additional DSA quests, summarised below. These are stated as planned content and are not present in the current codebase.

Quest	Concept
Crimson Mist Corridor	Sliding Window
Artillery Strike Zones	Prefix Sum
Titan-Breaker Siege	Binary Search on Answer
Fallen Army	Kadane's Algorithm

Beyond these named quests, further, less immediate expansion possibilities discussed in the project's design documentation and README include:

- Additional regions covering further DSA categories beyond the Plains of Origins.
- Algorithm-themed boss encounters marking the culmination of a region.
- Improved character/environment animation and the addition of music and sound effects.
- A larger Ancient Codex with richer per-entry content (worked examples, complexity comparisons).
- Additional automated and manual testing coverage, and general UI/gameplay polish.
- Longer-term, speculative ideas noted in the design documentation: achievements, cloud save, leaderboards, multiplayer, and custom maps. These are exploratory notes rather than committed roadmap items.

23. ADVANTAGES

- Active learning: the player must act on an algorithm's logic physically before writing code, rather than passively reading or watching an explanation.
- Visualisation of technique: the Two Pointer pillar puzzle gives a spatial, manipulable representation of an otherwise abstract index-narrowing process.
- Immediate application: the Code Trial is presented directly after the physical puzzle for the same technique, keeping the conceptual and implementation steps close together.
- Engagement through narrative: framing an algorithm as a story obstacle (a resonance barrier, ancient pillars) gives the learner a memorable context to associate with the technique.
- Connection between intuition and implementation: because the puzzle and the Code Trial operate on the same underlying values, succeeding at the puzzle directly informs how to reason about the code.
- Progress tracking: quest flags, the Ancient Codex, and the save system together give the player a persistent, visible record of what has been learned and completed.

These are design rationale points rather than claims of measured educational outcomes; no user study or learning-effectiveness evaluation has been conducted on Algoverse at this stage.

24. LIMITATIONS

- The project is still under active development; only one full quest (Two Pointers) is complete within the Plains of Origins, alongside the Dungeon's Binary Search trial.
- Content coverage is limited compared with mature, catalogue-scale DSA learning platforms, which offer many more problems and topics.
- Code execution depends on an external, internet-connected provider (Piston by default) that must be correctly configured through environment variables; without it, the Code Trial reports an “unconfigured” state rather than executing code.
- Save data is stored only in browser localStorage on a single device/browser profile, with no account system or cross-device synchronisation.
- No automated test suite exists for the frontend or backend code itself, so regressions in game logic or the execution pipeline currently rely on manual verification.
- Several UI/gameplay elements described in the project's design documentation (Inventory, Save Shrine, Victory/Game Over screens) are not yet implemented, so the currently playable experience ends after Quest 1 rather than reaching a defined narrative conclusion.
- The Code Trial editor is a plain text area rather than a full syntax-highlighting code editor, which limits editing convenience compared with dedicated coding-judge platforms.

25. CONCLUSION

Algoverse demonstrates that Data Structures and Algorithms concepts can be connected with interactive gameplay, physical visualisation, formal coding implementation, automated testing, and persistent progression within a single learning experience. The current implementation — spanning a Phaser/React frontend, a lightweight Node.js execution backend, and a remote sandboxed code-execution provider — delivers this loop end-to-end for two algorithms: Binary Search, introduced in the Dungeon, and the Two Pointer technique, taught through Quest 1 of the Plains of Origins.

The project's architecture — a data-driven challenge registry, a shared per-language harness system, and a versioned local save format — is structured to allow further quests to be added without redesigning the underlying systems, which directly supports the planned expansion of the Plains of Origins into additional DSA topics. As a B.Tech engineering project, Algoverse illustrates the practical integration of a 2D game engine, a React-based UI layer, a minimal backend service, and a third-party code-execution API into a single cohesive educational application, while leaving clearly identified future work — additional quests, expanded UI states, and automated testing — for continued development.

26. REFERENCES

- React — Official Documentation. <https://react.dev>
- Phaser — Official Documentation. <https://phaser.io/docs>
- Vite — Official Documentation. <https://vite.dev>
- Node.js — Official Documentation. <https://nodejs.org/docs>
- Piston — Code Execution Engine API. <https://github.com/engineer-man/piston>
- Judge0 — Online Code Execution System Documentation. <https://judge0.com>
- Git — Official Documentation. <https://git-scm.com/doc>
- GitHub Docs. <https://docs.github.com>
- ESLint — Official Documentation. <https://eslint.org/docs/latest>